V2 Integrated Gameplay Prototype Documentation| HKU

Date: 12/07/25 **AND** 19/06/26
Name: R de Fost
Project: Design Patterns for GDV - Game Architecture
Github: Design-Pattern-Exercise

- **Assignment**
  Select a game concept *with a* ***"relatively complex" gameplay system.***
  The system should support gameplay, e.g.: climbing / locomotion, ability systems, gunplay (aim-assist, generation, damage, etc.), artificial intelligence, level generation, character customization, etc. ***and contain at least 3 design patterns*** *(excl. singletons).*
  Develop a working prototype *of this game concept, that includes game startup, playable gameplay-loop, and a gameplay end-state (game-over, level complete, etc.)*
  Document *design decisions,* ***motivating*** *selection of patterns & integration.*

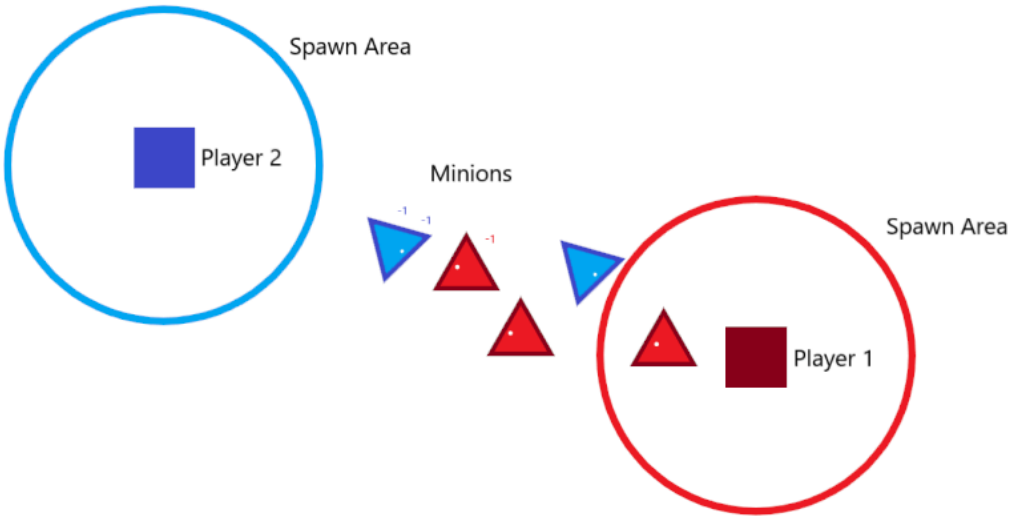
  RESTRICTION: The entire code-structure may only inherit a single MonoBehaviour. *This will force you to implement ways to integrate the system, and make explicit your approach to object communication.*

Table of contents

Game concept

A simple Local PVP game, where the players can cast spells to summon minions for them.
The players themself can’t attack/defend/heal themself but they can summon minions with dynamic spell combination to help them in these situations and kill the other player.

The main focus of the game is a simple minion summoning system where the player can combine inputs to create a minion that has a combination of the predefined effects / behaviours.



How To Play

	Movement	Casting	Cast Spell
Player 1	Keyboard: [W,A,S,D]	Keyboard: [I ,J, K, L]	Keyboard: [SPACE]
Player 2	Arrow Keys: [↑, ↓, ←, →]	Keypad: [5, 1, 2, 3]	Keypad: [ENTER]

Universal	**Left-shift**	*Displays available spells*

Movement: By pressing the movement inputs, the player can move around the screen to get in position and evade enemy minions.

Casting: By entering [4] casting inputs, the player pre-loads a new minion spell with a unique ability and stats (damage, speed, etc).
Cast Spell: To Cast a spell, the player presses the Cast Spell input.

Objective
By creating Casting Spells, minions are created with different stats and abilities, these will target the opponent, but will also attack any intersecting enemy minions, so they can also be used defensively.

Spell Creation *Spell list
To create a spell, the [casting input] are used. Each input has its own element attached to it. Each element has a unique effect on the different parts of the minion creation process.

- Upper input: Water element
- Lower input: Earth element
- Left input: Fire element
- Right input: Wind element

These elements have different effects based on the order that they are used in.

- Input[1]: [Ability]
- Input[2]: [Body]
- Input[3]: [Stability]
- Input[4]: [Amount]

Ability *Not yet implemented

	Ability
Water	Heals X amount based on Damage
Earth	Creates a obstacle at position*
Fire	Explodes dealing X amount of damage
Wind	Increases movement of players in the area

Body

	Damage	Defense	Speed
Water	4	4	5
Earth	4	6	3
Fire	6	2	5
Wind	2	4	7

Stability *Not yet implemented

	Activate Ability	Timer
Water	On spawning	2 sec
Earth	On spawning	5 sec
Fire	On Dying	2 sec
Wind	On Dying	5 sec

Amount *Not yet implemented

	Amount	Size	Area of Effect
Water	4 minions	0.8x	1.4x
Earth	1 minions	1.2x	1.0x
Fire	2 minions	1.0x	1.4x
Wind	4 minions	0.8x	1.4x

Gameplay systems

GameBehaviour: The Gamebehaviour is an abstract base class that imitates the functionality of the unity monobehaviour. It's a system that subscribes itself to the GameMaster Update function. Because of this, any GameBehaviour can have its own logic tied to the base update function without having to be called by any other class.

MultiServiceLocator: The MultiServiceLocator is a static class that keeps a reference to a given class that subscribes itself to it. When a second instance of the same class tries to subscribe itself to it will override the original class with itself, making sure that there will only be one Service class that can be called per class.
The MultiServiceLocator can be accessed anywhere in the code base, meaning that any class can request access to these subscribed classes, so they need to be used only for universal systems and with limited access.

CollisionSystem: The collisionSystem keeps track of all current active CollisionConponents. When the CollisionComponents are initialized, They get a reference to the CollsionSystem and subscribe themselves to the collider list.
In the collisionSystem every frame the Update() is called and checks for any collisions and sets the colliding object(s) to the collisionComponent.

ObjectPool: The ObjectPool is a system that when initiated, binds itself to a class with the IPoolable interface. This objectPool will keep track of a list of objects that can be activated and deactivated. This is done to

reduce the amount of times an object will have to be created to have them already loaded until needed by the class that its attached to.

MinionCreator: The MinionCreator is a system that when created takes in a list of input to use in its InputQueue. When this InputQueue is activated, it sends that list to the MinionCreator to reference with an internal dictionary to set the correct values to the minion based on the given input.
The minion values by calling decorators that based on the given input set the correct values and abilities to the minion.

InputHandler: The input handler keeps track of anty input that comes from the user. By calling the BindInputToCommand() function, the script that the InputHandler is attached to, can bind a needed input to react to a given command class.
The command class will be executed the moment that a bounded input is pressed.

InputQueue: The InputQueue is a system that uses the InputHandler to store any given inputs that have been set beforehand. The inputQueue checks for any ICommands that are IQueueable and will ignore any other inputs. This is used to check a set amount of inputs to have effect activation not only based on the current input, but a consecutive one.

Selection & Motivation

- Command pattern:
  When starting the project, the main feature would revolve around using input to create dynamic input. Because of its importance to the game, this part has a big chance of getting redesigned a lot. By knowing this, I wanted to make a system that would let me easily change the input and functionality of the spells.

- ObjectPool:
  In the gameplay, the players create minions to fight for them. Because the gameplay is designed to be relatively fast paced, these minions will quickly be destroyed after spawning them.

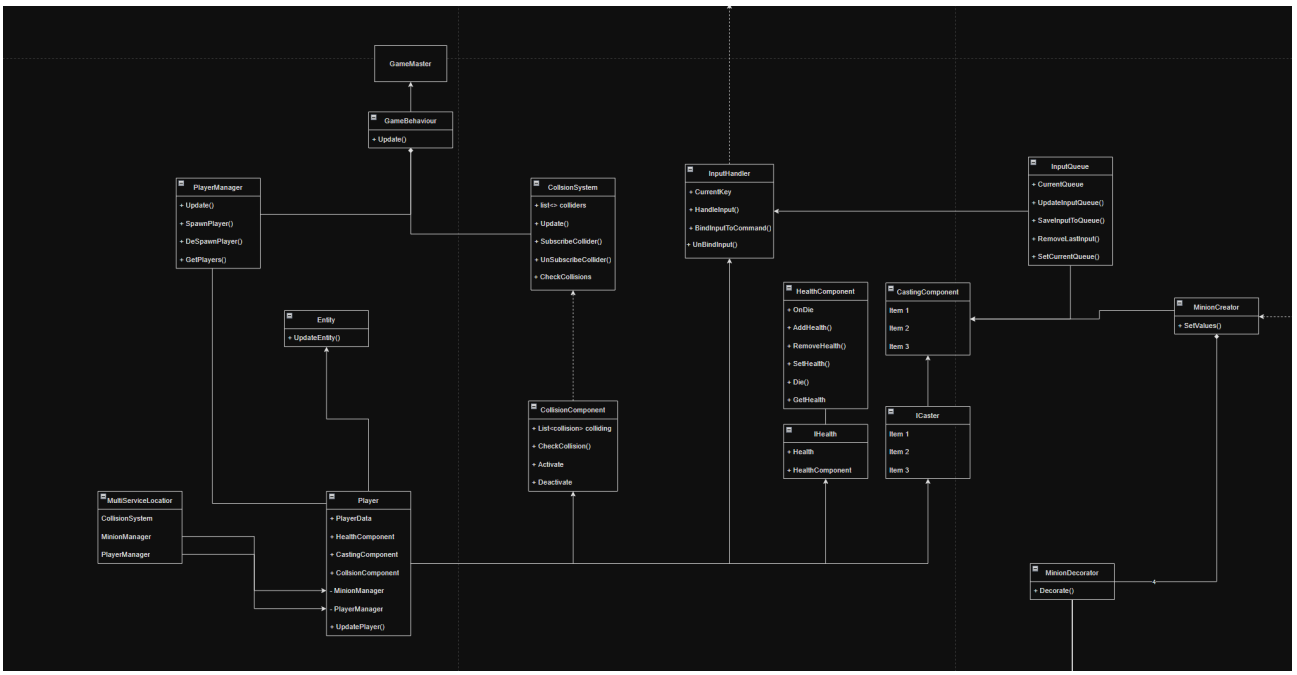
Knowing this, the objectPool is a great solution to make sure that not every second a new object has to be created. Besides this, the minions also have a lot of static components that can be retained between spawning and despawning, making the use of the objectpool more efficient.

- Decorator:
  In the game, the main focus is the use of dynamically created minions to fight your opponent. Because of this it will not be known what the contents of the variables from the minions will be until right before spawning them. Because of this a decorator is used to get the correct information and variables to be assigned based on the input that is given. Besides this it also helps with separating code. With a main class that combines all the needed decorators, this class will not be needed to change and only this information inside the decorators needs to be changed if needed.

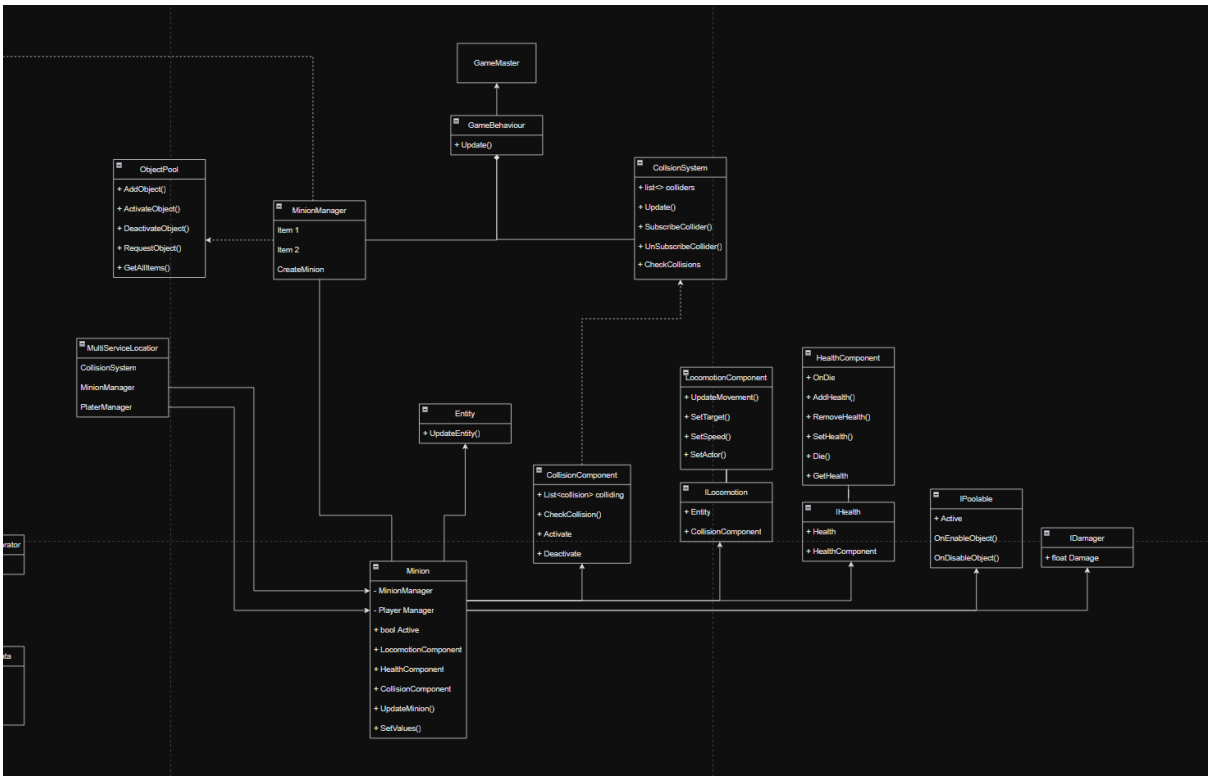
- Multi Service Locator:
  While working on the project there were a few systems that needed to be globally accessed. To fix this my initial solution was to use a Singleton Interface for needed classes. Later in the project is when this was replaced with a Multi Service Locator. This was done to have a more universal system that would store all needed information inside one class instead of being spread around different Singletons. while also making it so that there will always be one class to interact with and no accidental dubble assignments.

UML

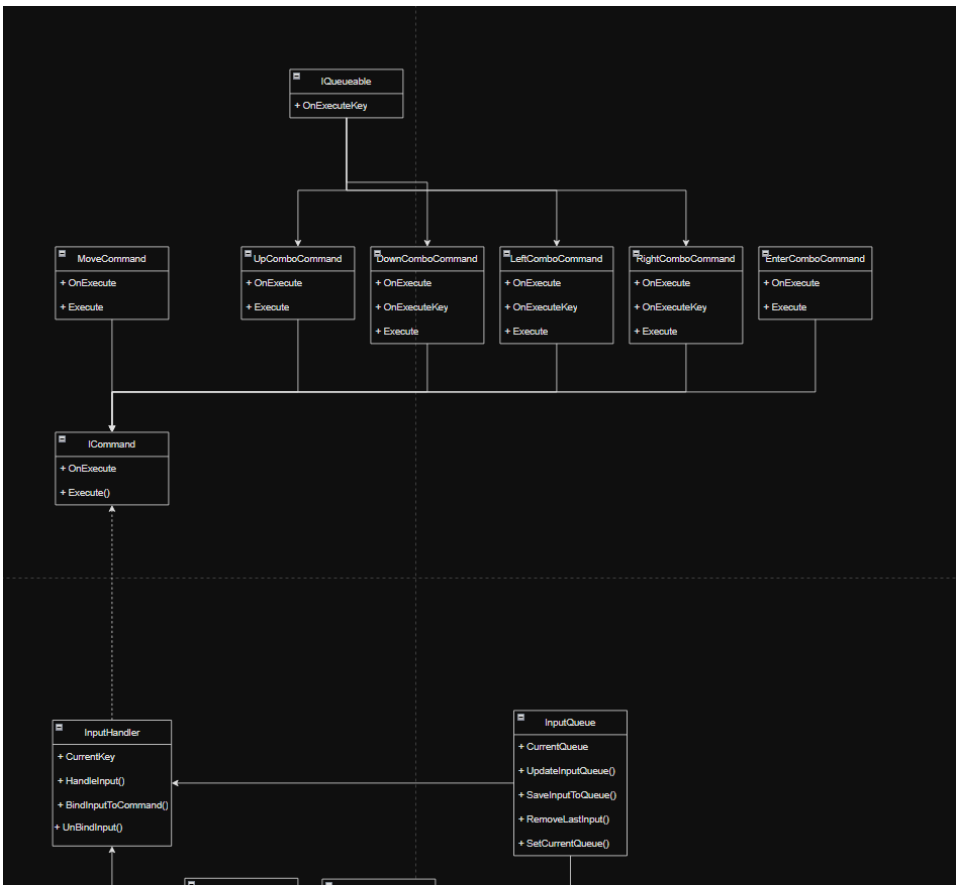
Player:



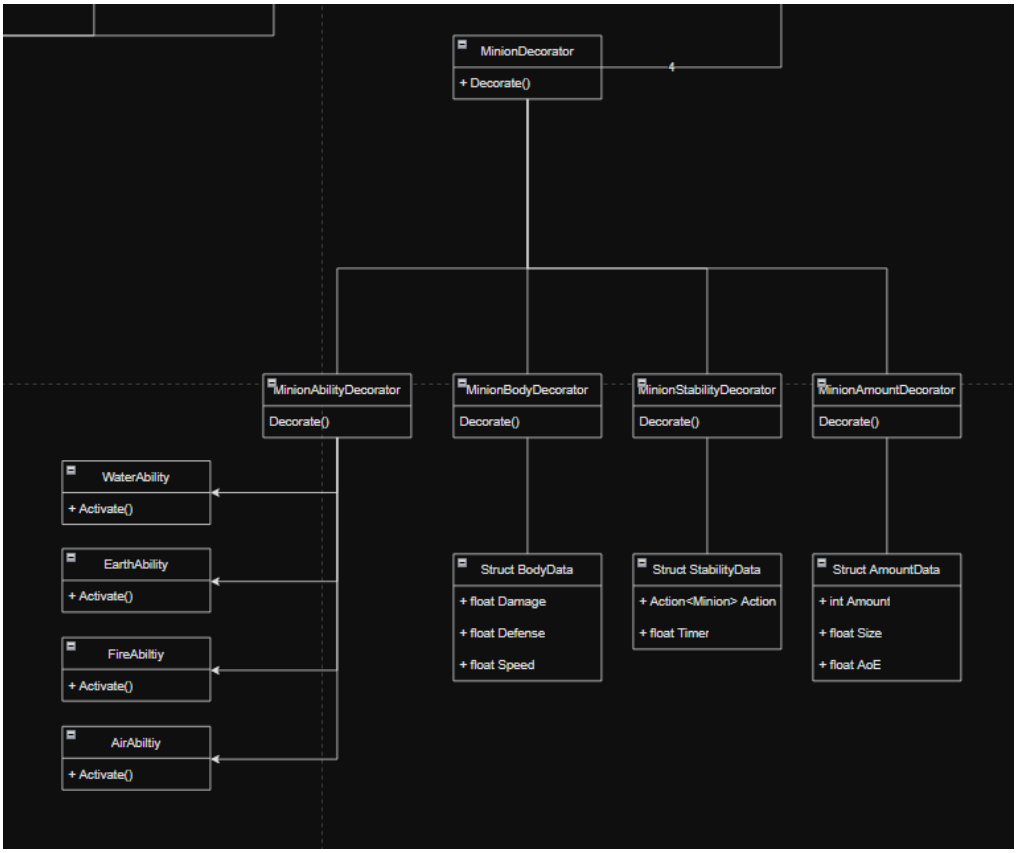
Minion:



InputHandler:


IQueueable
+ OnExecuteKey
MoveCommand
+ OnExecute
+ Execute
UpComboCommand
+ OnExecute
+ Execute
DownComboCommand
+ OnExecute
+ OnExecuteKey
+ Execute
LeftComboCommand
+ OnExecute
+ OnExecuteKey
+ Execute
RightComboCommand
+ OnExecute
+ OnExecuteKey
+ Execute
EnterComboCommand
+ OnExecute
+ Execute
ICommand
+ OnExecute
+ Execute()
InputHandler
+ CurrentKey
+ HandleInput()
+ BindInputToCommand()
+ UnBindInput()
InputQueue
+ CurrentQueue
+ UpdateInputQueue()
+ SaveInputToQueue()
+ RemoveLastInput()
+ SetCurrentQueue()

Decorator:


MinionDecorator
+ Decorate()
4
MinionAbilityDecorator
Decorate()
MinionBodyDecorator
Decorate()
MinionStabilityDecorator
Decorate()
MinionAmountDecorator
Decorate()
WaterAbility
+ Activate()
EarthAbility
+ Activate()
FireAbiltiy
+ Activate()
AirAbiltiy
+ Activate()
Struct BodyData
+ float Damage
+ float Defense
+ float Speed
Struct StabilityData
+ Action<Minion> Action
+ float Timer
Struct AmountData
+ int Amount
+ float Size
+ float AoE

Addendum (assignment: June 15, 2026)

Feedback

For the last assignment, The final product mainly missed integrated gameplay elements, such as feedback on the game state and a definitive end state to the game.

given feedback:

“De game runt, maar het is door het niveau van uitwerking van de feedback moeilijk in te schatten wat de geïmplementeerd systemen eigenlijk aan het doen zijn. De gameplay lijkt aanwezig, maar het systeem heeft geen duidelijk gekaderde gameplay loop (start, spelen, iemand wint, exit/opnieuw bijv.). Het is nog nodig om een minimale hoeveelheid feedback UI (zoals health, bv. als getal) en bv. on-screen buttons voor wat je kan doen (zodat je weet wat je in kunt drukken om minions te spawnen), plus dat de game wel aangeeft dat er iemand gewonnen heeft, en je dan de optie geeft om (eventueel) opnieuw te spelen of af te sluiten. Replay is niet per se noodzakelijk, maar feedback dat er iemand wint wel.”

Reworks

With the given feedback, I’ve looked at some of the lacking systems and feedback elements and set my focus on a few parts:

<u>Reworked Spells:</u> With the first iteration of the game, the idea was that the spell system was extremely modular, so that the player could create many different types of spell combinations. However, after looking back, I’ve come to the conclusion that this makes the creation of spells overly confusing.
To counter this, now there is an internal system that curates what combination counts as a valid spell. This still makes it easy to add and remove spells to the game, It also keeps the spell selection curated and easier to understand.

Spell List

Name	Input	Effect
Spell of Healing	↑/↓/↑/↑	*Spawns minions that heals the player on impact.*
Spell of Strength	↓/←/→/→	*Spawn a group of minions that deal a lot of damage.*

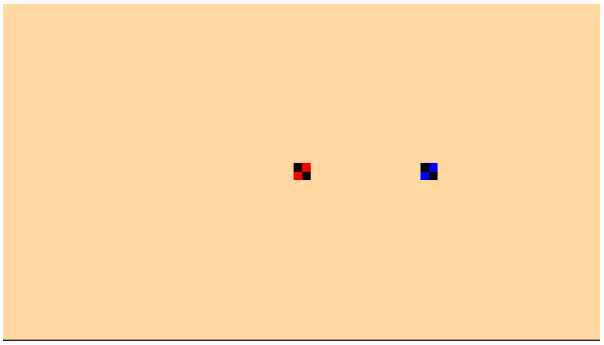
Spell of Areal Damage	←I←I→I←	*Spawn a group of minions that deal a medium amount of damage.*
Spell of Speed	→I→I←I←	*Spawns minions that speeds up the player on impact.*

UI Elements:

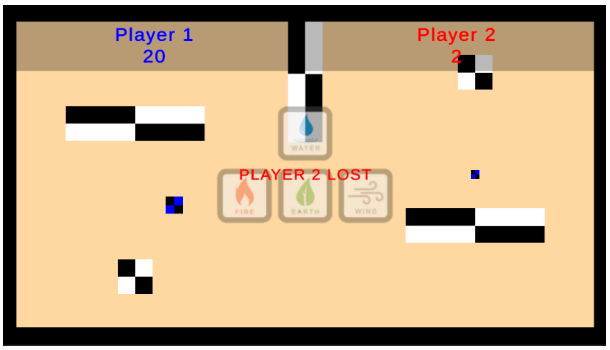
Another point of confusion was the lack of visual feedback, This was in the form of Player health, Possible input and win state.

Because of this limitation, I've written a new system that initializes the canvas and UI elements from within its own system. and added listeners to the dynamic UI elements to update them on player damage or game state.

Before

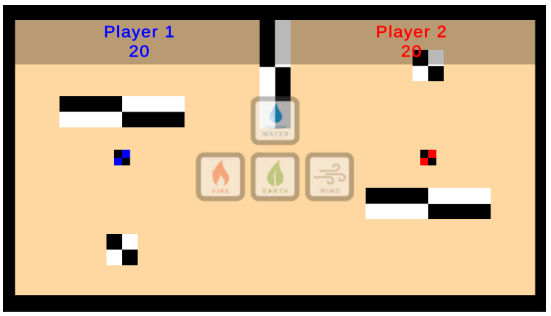


After



Environment:

Another point was that the gameplay did not have any context to be placed inside. So there is a simple levelRef that initializes a level inside the game and subscribes any colliders to the collision system.



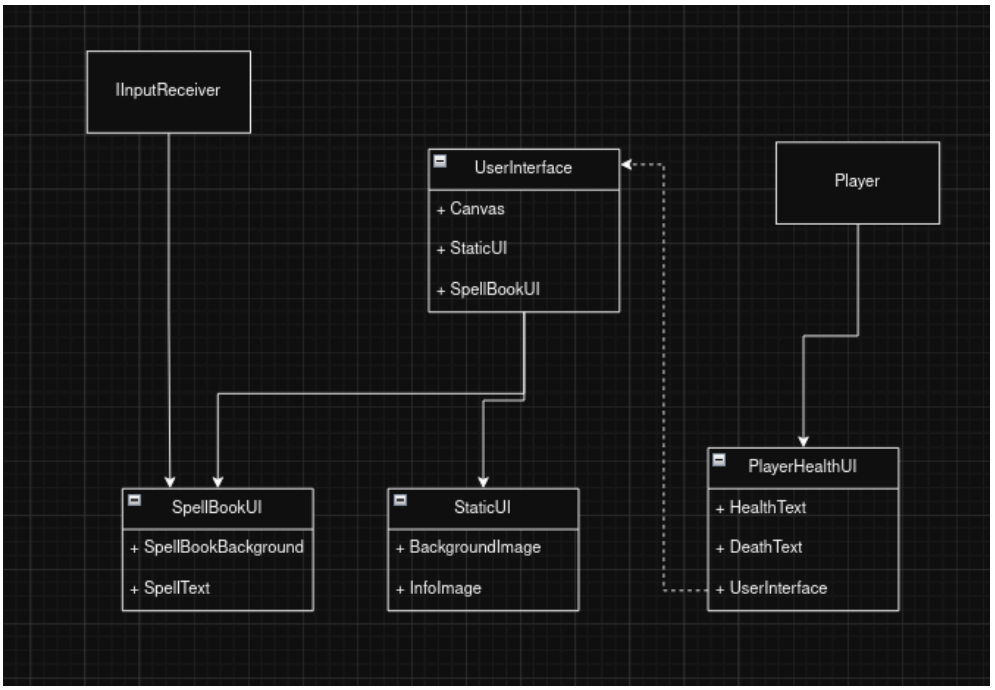
Added Gameplay Systems

User Interface: A gameBehaviour that handles the base initialization of UI elements and sets up the basis for any User interface elements through having the basis for creating and removing panels, images and text elements. It is subscribed to the multiserviceLocator for easy access to any needed data and holds the information for any static UI elements.

Level Ref:
A simple code structure that is initialized in the gameManager. It loads any environmental objects and obstacles on the correct locations.

UML

User Interface



Code Conventions

Line length

Code lines are not allowed to exceed **120 characters** so that code will always be readable on most screen sizes.

File Names

Every **file** and **folder** name is written in UpperCamelCase.

Namespaces

The namespace name is written in UpperCamelCase and is based on the folder name the script is placed in. Every class needs to be inside of a namespace.

```
namespace ExampleNamespace
{
    public class ExampleScript : MonoBehaviour
    {

    }
}
```

Classes

The Class name is written in UpperCamelCase.

```
public class ExampleScript : MonoBehaviour
{
    private void OnEnable()
    {
    }
}
```

Functions

The Function name is written in UpperCamelCase. Public functions need to have a summary or command.

```
private string ExampleFunction(string parameter)
{
    string newString = $"test {paramter}";
    return newString;
}

/// <summary>
/// A summary explaining the function of the function.
/// </summary>
public void SecondExampleFunction
{

}
```

When there is only 1 line inside of an function you can use a lambda expression.

```
private void ExampleFunction() => SecondExampleFunction();
```

If statements

If statements are always tabbed, except for return statements.

```
if (_exampleBoolean)
{
        if (isFalse) return

        ExampleFunction();
}
else
{
    SeccondExampleFunction();
    ThirdExampleFunction();
}
```

For loops

For loops are written using the same rules as `if` statements.

```
for (int i = 0; i < _exampleList.Count; i++)
{
   ExampleFunction(_exampleList[i]);
   SecondExampleFunction(_exampleList[i]);
}
```

For Each are written using the same rules as `for` loops.

```
foreach(GameObject gameObject in ExampleList)
{
   ExampleFunction(gameObject);
   SecondExampleFunction(gameObject);
}
```

Variables

As a general rule of thumb almost all variables are private and when access is needed use a Getter & Setter.

Global variable names are written in lowerCamelCase. and when a global variable is uses, alway use ‘this.’

```
private Object variableExample;

private void SecondExampleFunction(float variableExample)
{
        this.variableExample = variableExample
}
```

Readonly variable names are written the same as public variables so in lowerCamelCase.

```
public readonly Object variableExample;
```

Constant variable names are written in FULL_CAPITALS.

```
public const int EXAMPLE_CONSTANT_VALUE;
```

Temporary variables inside of an function always need to be written out and are written in lowerCamelCase.

```
private void ExampleFunction()
{
```

```
    float temporaryFloat = 1f;
    int temporaryInt = 1;
}
```

Temporary constants inside of an function always need to be written out and are written in FULL_CAPITALS.

```
private void ExampleFunction()
{
    const float TEMPORARY_FLOAT = 1f;
    const int TEMPORARY_INT = 1;
}
```

Property names are written in UpperCamelCase.

```
public int ExampleInteger
{
    get => _exampleInterger;
    set
    {
        if (value < 0)
            _exampleInterger = 0;
    }
}
```

Events

The **Action** name is written in UpperCamelCase and will almost always start with 'On'. **Unity Events** are written the same way as a `Action`.

```
public Action OnExampleEvent;
```

Structs

The struct name is written in UpperCamelCase and everything inside the struct follows the usual code conventions.

```
public struct ExampleStruct
{
    public double x;
    public double y;
}
```